



UNIVERSITY OF LEICESTER

School of Computing and Mathematical Sciences, University of Leicester

CO3219 Internet and Cloud Computing

Individual Report

Raul Blanco Vazquez – rbv3

Student Number: 239034344

12/2025

Table of Contents

Table of Contents.....	
1. Introduction	
2. Our Private Cloud	
Task 1: Architectural Design and Technology	
Task 2: Infrastructure Layers.....	
GCP Virtual Machines	
Network Security Configuration	
Load Balancing and Session Persistence.....	
Unmanaged Instance Groups: Stateful Infrastructure	
Kubernetes High Availability	
Horizontal Pod Autoscaler	
Task 3: Distributed Whiteboard Application	
Project Structure	
Application Architecture	
Client-Side Implementation	
State Management and New User Synchronisation.....	
Horizontal Scaling with Redis/Socket.IO	
Consistency Model.....	
Persistence and Durability	
Containerisation.....	
Observability and Monitoring.....	
3. Individual Contributions.....	
4. Verification and Testing	
5. Conclusion	
References	
Appendix: Figures	

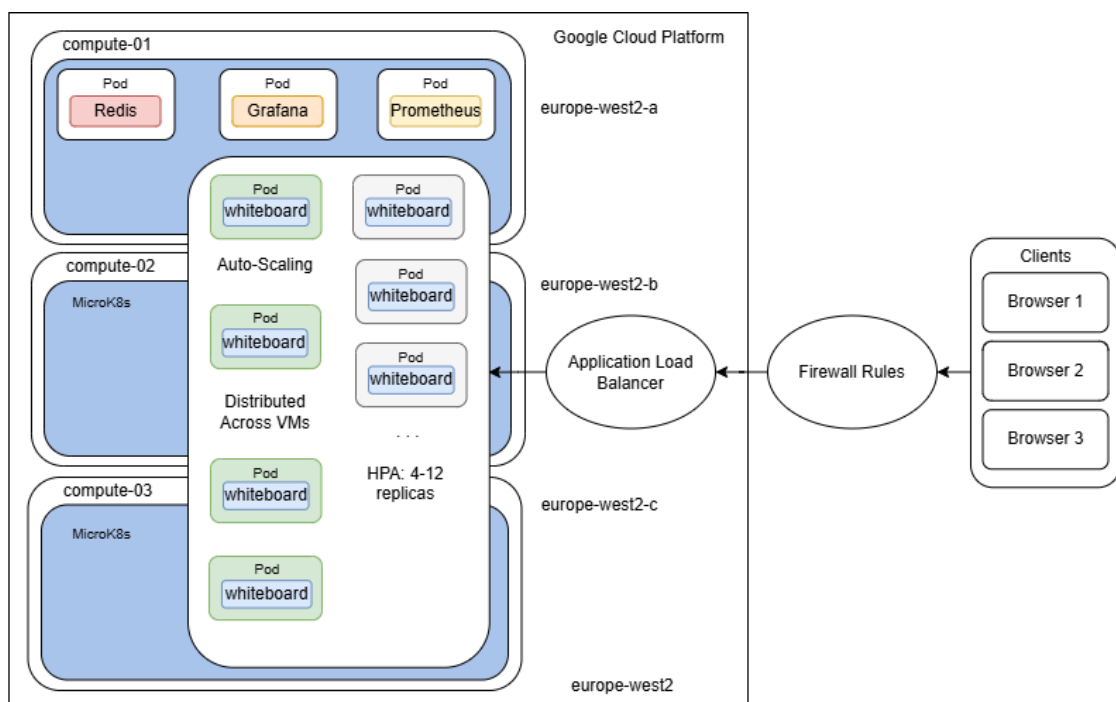
1. Introduction

This report presents my understanding of our group's solution for our CO3219 Coursework, examining how theoretical distributed systems principles were applied to create a fault-tolerant, scalable whiteboard application. The project demonstrates a practical implementation of cloud computing concepts including the CAP theorem trade-offs, consensus protocols for high availability, eventual consistency models, and horizontal scaling strategies [1].

Our work covered three main areas. First, we tackled the architectural design, figuring out how to meet requirements around consistency, scalability, and quality of service. Then came the infrastructure side, where we set everything up on Google Cloud Platform with container orchestration. Finally, we built and deployed a distributed real-time application that puts all these ideas into practice. Throughout this report, I break down how the different pieces work together from a distributed systems perspective, highlighting my own contributions.

2. Our Private Cloud

Task 1: Architectural Design and Technology



The architecture diagram illustrates our private cloud design. Technology choices were driven by the coursework requirements for consistency, scalability, fault tolerance, and quality of service:

- **GCP** provides multi-zone availability for fault isolation, integrated load balancing with session affinity for WebSocket support, and health checking for automatic failover.
- **MicroK8s** is a lightweight Kubernetes distribution that comes with high availability built in through dqlite consensus. This lets us do horizontal pod autoscaling, and when something goes wrong, pods get rescheduled automatically to keep things running.
- **Redis** addresses consistency through single-threaded total ordering, real-time pub/sub for cross-pod synchronisation, and AOF persistence for durability.
- **Socket.IO with Redis Adapter** makes it possible for clients and servers to talk back and forth in real time. When something happens, the event gets pushed out to every pod in the cluster, it doesn't matter which pod a particular client happened to connect to.

This design prioritises availability and partition tolerance per the CAP theorem [2], accepting eventual consistency. The following sections detail the implementation (Task 2) and distributed whiteboard application (Task 3).

Task 2: Infrastructure Layers

GCP Virtual Machines

Fault tolerance in distributed systems requires understanding failure domains, the blast radius of potential failures [3]. Our infrastructure implements fault isolation through GCP's availability zone architecture. I deployed Three Compute Engine VMs across separate zones (europe-west2-a, europe-west2-b, europe-west2-c), each representing an independent failure domain with separate power, cooling, and network infrastructure (Fig. 1). This design ensures that zone-level failures, whether from power outages, network partitions, or hardware failures, affect at most one-third of compute capacity.

```
kathan@compute-01:~$ sudo microk8s kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
compute-01    Ready     <none>    4d13h v1.32.9
compute-02    Ready     <none>    169m  v1.32.9
compute-03    Ready     <none>    176m  v1.32.9
```

Each VM runs Ubuntu 22.04 LTS on n2-custom-4-8192 machine type (4 vCPUs, 8GB memory) with 25GB balanced persistent disk. The VPC network configuration enables internal routing between nodes while firewall rules control external access, implementing defence-in-depth security by restricting traffic to necessary ports.

Network Security Configuration

The firewall configuration segments traffic by purpose, restricting cluster communication to the internal subnet while enabling required external access. The allow-microk8s-internal rule restricts cluster communication to the internal subnet, preventing external access to Kubernetes API (16443), dqlite consensus (19001), and kubelet (10250-10259) ports, critical for cluster security (Fig. 2).

Load Balancing and Session Persistence

Load balancing distributes client requests across backend servers to improve throughput and availability [1]. The GCP Application Load Balancer implements Layer 7 (HTTP) load balancing with health-aware routing. Understanding the configuration requires examining how WebSocket applications interact with load balancers.

Socket.IO, used for real-time communication, initially establishes connections via HTTP long-polling before upgrading to WebSocket. Without session affinity, sequential polling requests would be distributed across different backend pods, each unaware of the connection state established by others, causing connection failures. The backend service configuration addresses this (Fig. 3)

Session affinity uses generated cookies to implement "sticky sessions", ensuring all requests from a client reach the same backend until the connection upgrades to WebSocket. The health check enables rapid failure detection, removing unhealthy backends within 10-15 seconds (Fig. 4).

Unmanaged Instance Groups: Stateful Infrastructure

The load balancer distributes traffic to three unmanaged instance groups (one per zone). Unmanaged groups were chosen over managed instance groups because our VMs maintain stateful configurations that cannot be captured in instance templates. Managed groups provide auto-healing by recreating failed VMs from templates, but a recreated VM would lack MicroK8s installation, cluster membership, dqlite consensus participation, and locally-imported container images. Kubernetes provides application-level self-healing through pod rescheduling; VM-level recreation would conflict with this model [3] (Fig. 5).

Kubernetes High Availability

High availability in distributed systems requires consensus protocols to maintain consistent state across replicas despite failures [4].

I implemented MicroK8s with its high-availability mode, which distributes the control plane across all nodes using dqlite, a distributed SQLite implementation based on the Raft consensus protocol [4]. Raft ensures that cluster state remains consistent across nodes through leader election and log replication. With three nodes, we tolerate any single failure while maintaining consensus:

```
kathan@compute-01:~$ sudo microk8s status
microk8s is running
high-availability: yes
  datastore master nodes: 10.154.0.6:19001 10.154.0.7:19001 10.154.0.5:19001
  datastore standby nodes: none
```

Each node participates as a dqlite voter (Role 0 in cluster.yaml), meaning any node can become leader if the current leader fails. This transforms the failure model: rather than single-master topology where master failure halts all management operations, any two nodes can maintain full cluster functionality including scheduling, scaling, and recovery:

```
kathan@compute-01:~$ sudo cat /var/snap/microk8s/current/var/kubernetes/backend/cluster.yaml
- ID: 3297041220608546238
  Address: 10.154.0.6:19001
  Role: 0
- ID: 14226017189575688149
  Address: 10.154.0.7:19001
  Role: 0
- ID: 15464382521432144559
  Address: 10.154.0.5:19001
  Role: 0
```

Horizontal Pod Autoscaler

Cloud elasticity (the ability to dynamically adjust resources to match demand) is achieved through horizontal scaling (adding/removing instances) rather than vertical scaling (increasing instance size) [5]. The Kubernetes Horizontal Pod Autoscaler (HPA) implements reactive autoscaling based on observed metrics:

```
kathan@compute-01:~$ sudo microk8s kubectl get hpa
NAME          REFERENCE          TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
whiteboard    Deployment/whiteboard  cpu: 1%/50%    4         12        4          2d16h
```

The HPA control loop follows a proportional scaling algorithm: when average CPU utilisation exceeds 50%, replica count increases; when it drops below, replicas decrease. Resource requests on pods enable metrics-server to calculate utilisation percentages. Without requests, HPA cannot determine when to scale. I configured this HPA with bounds of 4-12 replicas to balance responsiveness against resource constraints.

Task 3: Distributed Whiteboard Application

The whiteboard application runs across multiple nodes with each pod capable of independently serving users while maintaining a consistent, shared canvas. The following subsections explain the application's design and implementation in terms of consistency, replication, and state management.

Project Structure

The whiteboard application consists of four files organised as follows:

```
whiteboard-redis/
├── Dockerfile           # Container build instructions
├── package.json         # Dependencies and scripts
├── server.js            # Express + Socket.IO server
└── public/
    └── index.html       # Canvas UI and client-side JS
```

The package.json defines the application dependencies:

```
// package.json
{
  "name": "whiteboard",
  "dependencies": {
    "express": "^4.18.2",
    "socket.io": "^4.6.1",
    "@socket.io/redis-adapter": "^8.2.1",
    "redis": "^4.6.7"
  }
}
```

The `@socket.io/redis-adapter` is the critical dependency enabling cross-pod synchronisation. It allows Socket.IO servers running on different pods to share events through Redis pub/sub channels.

Application Architecture

The whiteboard is a Node.js application using Express for HTTP serving and Socket.IO for real-time WebSocket communication. Four replicas run across three compute nodes, each serving users independently. The key architectural insight is that despite physical distribution across pods, all instances behave as a unified system. A user connected to pod A sees drawings from users connected to pods B, C, and D in real time.

This is achieved through the Socket.IO Redis adapter, which uses Redis pub/sub channels to broadcast events across all connected Socket.IO server instances. When any pod receives a draw event from a client, it publishes to Redis; all other pods receive via subscription and broadcast to their locally connected clients.

Client-Side Implementation

The frontend uses HTML5 Canvas for drawing with JavaScript handling user input and Socket.IO for server communication. When a user draws, the client captures mouse events, renders locally for immediate feedback, and emits draw events to the server:

```
// Client-side drawing and synchronisation
canvas.addEventListener('mousemove', (e) => {
  if (!drawing) return;
  const data = { x1: lastX, y1: lastY, x2: e.clientX, y2: e.clientY, color:
colorPicker.value, size: brushSize.value };
  drawLine(data.x1, data.y1, data.x2, data.y2, data.color, data.size);
  socket.emit('draw', data);
});
// Receive drawings from other users (via any pod)
socket.on('draw', drawLine);
```

Each draw event contains line coordinates (x1, y1 to x2, y2), colour, and brush size. The client both draws locally and emits to the server, while also listening for 'draw' events from other users to render their contributions.

State Management and New User Synchronisation

A critical requirement is that when a new user joins, they must see the current whiteboard state, including all existing drawings. I implemented this by storing every draw event in a Redis list. Ensuring that when a new user joins the system, they obtain the current state of the whiteboard and everything is displayed to every active user.

```
io.on('connection', async (socket) => {
  // Replay drawing history to new client
  const history = await pubClient.lRange(HISTORY_KEY, 0, -1);
  history.forEach(event => socket.emit('draw', JSON.parse(event)));
});
```

This approach treats Redis as an event log, so the current whiteboard state is derived by replaying all historical events. When new draw events arrive, they are appended to the

history and broadcast to other clients. Testing confirmed over 660 drawing events persisting across sessions.

Horizontal Scaling with Redis/Socket.IO

I ensured the application was designed to scale horizontally, by making the Redis adapter decouple pod-local Socket.IO instances from the global event stream. Adding more pods (via HPA or manual scaling) simply adds more subscribers to the Redis pub/sub channels. New instances automatically receive all events and can serve additional users without configuration changes. The stateless pod design (all state in Redis) means pods can be added or removed without data loss.

Consistency Model

The CAP theorem states that distributed systems can provide only two of three properties: Consistency, Availability, and Partition tolerance [2]. Since network partitions are inevitable, practical systems must choose between consistency and availability during partitions. Our whiteboard application prioritises availability with eventual consistency, meaning users can always draw (availability), but there may be brief delays before all users see changes (relaxed consistency).

The consistency mechanism uses Redis publish-subscribe messaging. Redis's single-threaded architecture ensures total ordering of operations within a single instance, preventing write conflicts [6]. The Socket.IO Redis adapter extends this to enable cross-pod event broadcasting:

```
// server.js - Redis adapter for cross-pod sync
const { createAdapter } = require('@socket.io/redis-adapter');
const pubClient = createClient({ url: 'redis://redis:6379' });
const subClient = pubClient.duplicate();

Promise.all([pubClient.connect(), subClient.connect()]).then(() => {
  io.adapter(createAdapter(pubClient, subClient));

  io.on('connection', async (socket) => {
    // Retrieve history for new clients (state reconstruction)
    const history = await pubClient.lRange(HISTORY_KEY, 0, -1);
    history.forEach(data => socket.emit('draw', JSON.parse(data)));

    socket.on('draw', async (data) => {
      // Persist to Redis list + broadcast via pub/sub
      await pubClient.rPush('whiteboard:history', JSON.stringify(data));
      socket.broadcast.emit('draw', data);
    });
  });
});
```

[See Fig. 6: Complete server.js source code in Appendix]

When any pod receives a draw event, it publishes to Redis; all other pods receive via subscription and broadcast to their local clients. This implements the publish-subscribe pattern for decoupled, scalable event distribution [6]. Drawing history in Redis lists enables

state reconstruction when new clients connect or pods restart, storing events as an append-only log from which current state can be derived.

Persistence and Durability

Durability (ensuring data survives failures) requires persistence to stable storage. Redis provides configurable persistence through append-only file (AOF) mode, where every write operation is logged before acknowledgment [6]:

```
# Redis deployment with AOF persistence
spec:
  containers:
    - name: redis
      image: redis:7-alpine
      args: ["--appendonly", "yes"]
      ports: [containerPort: 6379]
```

AOF persistence trades write performance for durability: each operation incurs disk I/O latency but survives process crashes. Testing confirmed over 660 drawing events persisting across pod restarts, demonstrating successful durability guarantees.

Containerisation

Containerisation provides consistent deployment environments through OS-level virtualisation, packaging applications with their dependencies [3]. The whiteboard application uses a Node.js Alpine base for minimal image size:

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

Without a container registry, images are distributed by building locally on each node and importing to MicroK8s's containerd runtime. This approach suits development environments; production deployments would use a registry for centralised image management.

Observability and Monitoring

Observability (the ability to understand system state from external outputs) is essential for operating distributed systems [1]. The monitoring stack implements the pull-based collection model where Prometheus scrapes metrics endpoints at configured intervals. Node-exporters expose system metrics (CPU, memory, disk, network) while metrics-server provides Kubernetes resource metrics for HPA decisions. Grafana visualises time-series data through configurable dashboards (Fig. 7).

3. Individual Contributions

- Selected GCP, MicroK8s and Redis as core technologies
- Configured the three GCP VMs across fault domains
- Installed MicroK8s on all nodes and networking
- Implemented the HA control plane configuration using dqlite consensus
- Configured HPA with resource requests for elastic scaling
- Configured NodePort services for external access
- Set up node-exporters on all 3 VMs for Prometheus metrics
- Set up application state management and new user synchronisation
- Set up application horizontal scaling with Socket.IO and Redis

4. Verification and Testing

System verification confirmed distributed systems properties:

```
kathan@compute-01:~$ sudo microk8s kubectl get nodes
NAME          STATUS    ROLES    AGE      VERSION
compute-01    Ready     <none>   6d15h    v1.32.9
compute-02    Ready     <none>   2d5h     v1.32.9
compute-03    Ready     <none>   2d5h     v1.32.9
kathan@compute-01:~$ sudo microk8s kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE    IP             NODE          NOMINATED NODE   READINESS GATES
redis-5f89fd8b68-bngzj    1/1     Running   0           2d5h   10.1.174.69    compute-03    <none>            <none>
whiteboard-bbb6f48cc-2xfdw 1/1     Running   0           2d5h   10.1.174.70    compute-03    <none>            <none>
whiteboard-bbb6f48cc-gm4qd 1/1     Running   0           2d5h   10.1.242.208   compute-01    <none>            <none>
whiteboard-bbb6f48cc-kqw2k 1/1     Running   0           2d5h   10.1.174.68    compute-03    <none>            <none>
whiteboard-bbb6f48cc-p4jnt 1/1     Running   0           2d17h  10.1.242.254   compute-01    <none>            <none>
```

Consistency was verified by simultaneous drawing from multiple browsers, and changes propagated within milliseconds via Redis pub/sub.

5. Conclusion

This project demonstrates the practical implementation of distributed systems principles within educational resource constraints. The architecture achieves fault tolerance through multi-zone VM deployment across three availability zones, high availability via MicroK8s dqlite consensus (Raft protocol), real-time synchronisation using Redis pub/sub with the Socket.IO adapter, and elastic scaling through Horizontal Pod Autoscaler (4-12 replicas).

The system prioritises availability and partition tolerance per the CAP theorem [2], accepting eventual consistency to ensure users can always interact with the whiteboard even during partial failures. Verification through simultaneous multi-browser testing confirmed sub-second drawing propagation across pods, validating that the consistency and replication mechanisms function as designed.

References

- [1] M. Armbrust et al., "A View of Cloud Computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50-58, 2010.
- [2] E. Brewer, "CAP Twelve Years Later: How the 'Rules' Have Changed," *Computer*, vol. 45, no. 2, pp. 23-29, 2012.
- [3] L. Abdollahi Vayghan et al., "Deploying Microservice Based Applications with Kubernetes," *2018 IEEE International Conference on Cloud Computing*, pp. 970-973.
- [4] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm," *2014 USENIX Annual Technical Conference*, pp. 305-319.
- [5] N. Herbst et al., "Elasticity in Cloud Computing: What It Is, and What It Is Not," *10th International Conference on Autonomic Computing*, pp. 23-27, 2013.
- [6] Redis Ltd., "Redis Pub/Sub and Persistence," *Redis Documentation*. <https://redis.io/docs/>

Appendix: Figures

Fig. 1: GCP VM Instances across availability zones

VM instances

Filter Enter property name or value

☐ Status	Name ↑	Zone	Recommendations	In use by	Internal IP	External IP	Connect
☐	compute-01	europe-west2-a	Save \$38/mo	whiteboard-group	10.154.0.6 (nic0)	34.89.20.180 (nic0)	SSH
☐	compute-02	europe-west2-b		whiteboard-group-2	10.154.0.5 (nic0)	34.142.20.240 (nic0)	SSH
☐	compute-03	europe-west2-c	Save \$49/mo	whiteboard-group-3	10.154.0.7 (nic0)	34.142.73.112 (nic0)	SSH

Fig. 2: Firewall rules configuration

VPC firewall rules

Firewall rules control incoming and outgoing traffic to an instance. By default, all incoming traffic to your network is blocked. [Learn more](#)

Note: App Engine firewalls are managed in the [App Engine firewall rules section](#).

SMTP port 25 disallowed in this project. [Learn more](#)

[Refresh](#) [Configure logs](#) [Delete](#)

Filter Enter property name or value

☐ Name	Type	Targets	Filters	Protocols/ports	Action	Priority	Network ↑
☐ allow-30443	Ingress	Apply to all	IP ranges: 0.0.0.0/0	tcp:30443	Allow	1000	default
☐ allow-4000	Ingress	nomachine	IP ranges: 0.0.0.0/0	tcp:4000	Allow	1000	default
☐ allow-grafana	Ingress	Apply to all	IP ranges: 0.0.0.0/0	tcp:3000, 9090, 30030, 30088, 30090	Allow	1000	default
☐ allow-microk8s-internal	Ingress	Apply to all	IP ranges: 10.154.0.0/24	tcp:9100, 10250, 10255, 10257, 10259, 12379, 16443, 19001, 25000 udp:4789	Allow	1000	default
☐ default-allow-health-check	Ingress	lb-health-check	IP ranges: 35.191.0.0/16, 130.211.0.0/22, 209.85.152.0/22,	tcp	Allow	1000	default
☐ default-allow-health-check-jvs	Ingress	lb-health-check	IP ranges: 2600:1901:8001::/48, 2600:2800:1:b029::/64	tcp	Allow	1000	default
☐ default-allow-http	Ingress	http-server,	IP ranges: 0.0.0.0/0	tcp:80	Allow	1000	default
☐ default-allow-https	Ingress	https-server,	IP ranges: 0.0.0.0/0	tcp:443	Allow	1000	default
☐ default-allow-icmp	Ingress	Apply to all	IP ranges: 0.0.0.0/0	icmp	Allow	65534	default
☐ default-allow-internal	Ingress	Apply to all	IP ranges: 10.0.0.0/24	tcp:0-65535 udp:0-65535 icmp	Allow	65534	default
☐ default-allow-udp	Ingress	Apply to all	IP ranges: 0.0.0.0/0	tcp:3389	Allow	65534	default
☐ default-allow-ssh	Ingress	Apply to all	IP ranges: 0.0.0.0/0	tcp:22	Allow	65534	default

Fig. 3: Load balancer backend configuration

Backend

Backend services

1. whiteboard-backend

Endpoint protocol	HTTP
Named port	http
Timeout	3,600 seconds
IP address selection policy	Only IPv4
Health check	whiteboard-health
Logging	Disabled
Backend security policy	None
Session affinity	Generated cookie
Affinity cookie TTL	86,400 seconds
Connection draining timeout	300 seconds
Traffic policy	Locality load balancing policy: Ring hash Circuit breakers: Disabled Outlier detection: Disabled View details
Identity-Aware Proxy	Disabled
Backend authentication	Disabled

[^ Hide advanced](#)

Backends

Name 	Type	IP stack type	Scope	Healthy	Auto-scaling	Balancing mode	Selected ports 	Capacity	Preference level 
whiteboard-group	Instance group	IPv4	europa-west2-a	 1 of 1	No configuration	Max backend utilisation: 80%	30088	100%	None
whiteboard-group-2	Instance group	IPv4	europa-west2-b	 1 of 1	No configuration	Max backend utilisation: 80%	30088	100%	None
whiteboard-group-3	Instance group	IPv4	europa-west2-c	 1 of 1	No configuration	Max backend utilisation: 80%	30088	100%	None

Fig. 4: Health check configuration

whiteboard-health

Region

europa-west2

In use by

[whiteboard-backend](#)

Path

/

Protocol

HTTP

Port

30088

PROXY protocol

None

Logs

Disabled

Interval

5 seconds

Timeout

5 seconds

Healthy threshold

2 consecutive successes

Unhealthy threshold

2 consecutive failures

Tags



Fig. 5: Unmanaged instance groups

Instance groups [Create instance group](#) [Refresh](#) [Delete](#)

Instance groups are collections of VM instances that use load balancing and automated services, like auto-scaling and auto-healing. [Learn more](#)

Filter Enter property name or value

Status	Name	Instances	Template	Group type	Creation time	Recommendation	Auto-scaling	Location	In Use By
☒	whiteboard-group	1	-	Unmanaged	Dec 4, 2025, 1:29:18 am UTCZ			europa-west2-a	whiteboard-backend
☒	whiteboard-group-2	1	-	Unmanaged	Dec 4, 2025, 1:30:25 am UTCZ			europa-west2-b	whiteboard-backend
☒	whiteboard-group-3	1	-	Unmanaged	Dec 5, 2025, 1:49:53 pm UTCZ			europa-west2-c	whiteboard-backend

Fig. 6: Complete server.js and index.html source

```
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');
const { createClient } = require('redis');
const { createAdapter } = require('@socket.io/redis-adapter');

const app = express();
const server = http.createServer(app);
const io = new Server(server, {
  transports: ['polling', 'websocket'],
  cors: { origin: '*', methods: ['GET', 'POST'] },
  pingTimeout: 60000, pingInterval: 25000
});

app.use(express.static(__dirname + '/public'));
const redisUrl = process.env.REDIS_URL || 'redis://localhost:6379';
const pubClient = createClient({ url: redisUrl });
const subClient = pubClient.duplicate();

Promise.all([pubClient.connect(), subClient.connect()]).then(() => {
  io.adapter(createAdapter(pubClient, subClient));
  console.log('Connected to Redis at', redisUrl);
}).catch(err => console.error('Redis error:', err));

const HISTORY_KEY = 'whiteboard:history';

io.on('connection', async (socket) => {
  try {
    const history = await pubClient.lRange(HISTORY_KEY, 0, -1);
    history.forEach(item => socket.emit('draw', JSON.parse(item)));
  } catch (err) {}

  socket.on('draw', async (data) => {
    socket.broadcast.emit('draw', data);
    try { await pubClient.rPush(HISTORY_KEY, JSON.stringify(data)); } catch (err) {}
  });

  socket.on('clear', async () => {
    socket.broadcast.emit('clear');
    try { await pubClient.del(HISTORY_KEY); } catch (err) {}
  });
});

server.listen(process.env.PORT || 3000);
```

Fig. 7: Grafana monitoring dashboard

